

Figure 1: Box plot for the easy query on Berlin Sparql Benchmark for Dense Vector Querying (DVQ) $n = 4.24 \cdot 10^4$. We compare the baseline (RDFTensor [2]) and Neo4j against our EVerok with and without a vocabulary and with and without indexing. We note that the time t is logarithmic.

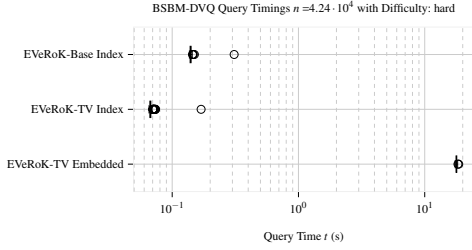


Figure 2: Box plot for the hard query on BSBM-DVQ $n = 4.2 \cdot 10^4$. We compare the baseline (RDFTensor [2]) against our EVerok with and without a vocabulary and with and without indexing. We note that the time t is logarithmic.

A Supplemental Material

This appendix supplements the main finding of the paper with additional statistical findings and efficiency analysis based on monitored hardware resources.

A.1 Statistical Information

First, we detail our timing results with additional, supporting, statistics for our core findings. We show all results that did not time out, beginning with the BSBM-DVQ.

BSBM-DVQ. Table 1 details all our measurements on BSBM-DVQ and indicates statistically faster timings of these speedups in **bold**. We also include the two-stage queries within the table, showing that it takes double the time compared to the baseline when using QLever, and up to $30\times$ longer when using plain Fuseki, not even running without timeouts beyond the lower sizes of $m \leq 1000$. The table also shows limitations in our acquisition methodology, as we were not able to gather speedup results beyond the lower m , as the Fuseki implementation timed out. EVerok with the tensor vocabulary is also significantly faster on all benchmarked combinations. The cold start speedup, i.e. the time the first query takes on a completely cold start, without indices being build, is also shown in the

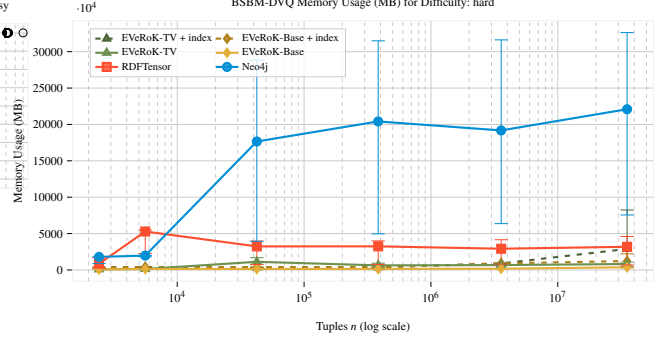


Figure 3: Median memory usage for the hard query on the BSBM-DVQ of increasing total tuples n . We compare the baseline (RDFTensor [2]) and Neo4j against our EVerok with and without a vocabulary and with and without indexing. The shaded areas indicate the distribution within the percentiles.

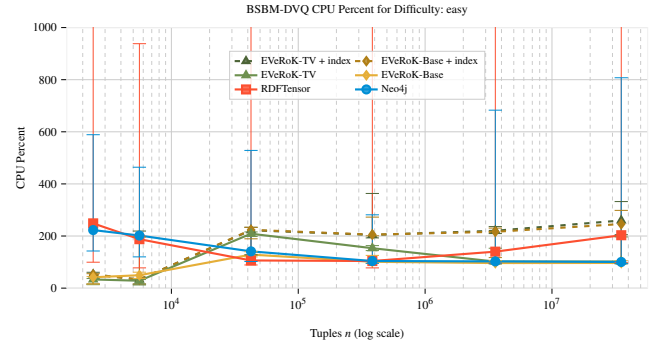


Figure 4: Median CPU usage for the easy query on the BSBM-DVQ of increasing total tuples n . We compare the baseline (RDFTensor [2]) and Neo4j against our EVerok with and without a vocabulary and with and without indexing. The shaded areas indicate the distribution within the percentiles.

last columns. These indicate significant, but only minor differences for the base and tensor vocabulary cases, with larger differences in the indexed query versions due to the index build times. Further timing distributions supporting our results can be read from Figures 1 and 2. EVerok shows a strong lead, also with very tight distributions around the medians compared to the other implementations. The two-stage result is naturally slower, as it uses the same pipeline for all systems. The hard queries in Figure 2 show that the base and tensor vocabulary indices have also a tight distribution, with just one outlier – the initial index build.

Our resource usage also indicate strong efficiency gains in our systems, as the memory usage of Figure 3 show a strong lead of EVerok, since the other compared systems are Java-based, requiring comparatively large amounts of memory just for the Java Virtual

Table 1: Speedup of EVerRoK compared to RDFTensor for our BSBM-DVQ. The time is the median time over the group, significant speedups are indicated in bold. Query combinations that timed out are omitted.

Difficulty	Query Type	Engine	m	t (s)	Speedup (vs Baseline)	Speedup (vs Cold Start)
easy	Embedded	Neo4j	1	$0.39 \cdot 10^{-1}$	$0.37 \cdot 10^0$	$0.28 \cdot 10^{-1}$
			10	$0.44 \cdot 10^{-1}$	$0.64 \cdot 10^0$	$0.52 \cdot 10^{-1}$
			100	$0.81 \cdot 10^{-1}$	$1.77 \cdot 10^0$	$0.23 \cdot 10^0$
			1000	$0.37 \cdot 10^0$	$4.10 \cdot 10^0$	$1.43 \cdot 10^0$
			10000	$3.78 \cdot 10^0$	$4.25 \cdot 10^0$	$3.58 \cdot 10^0$
		RDFTensor	1	$0.16 \cdot 10^{-1}$	1	$0.27 \cdot 10^{-1}$
			10	$0.30 \cdot 10^{-1}$	1	$0.39 \cdot 10^{-1}$
			100	$0.16 \cdot 10^0$	1	$0.12 \cdot 10^0$
			1000	$1.48 \cdot 10^0$	1	$0.45 \cdot 10^0$
			10000	$1.62 \cdot 10^1$	1	$0.78 \cdot 10^0$
		EVerRoK-Base	1	$0.47 \cdot 10^{-2}$	$2.87 \cdot 10^0$	$1.52 \cdot 10^0$
			10	$0.13 \cdot 10^{-1}$	$2.00 \cdot 10^0$	$1.47 \cdot 10^0$
			100	$0.85 \cdot 10^{-1}$	$1.61 \cdot 10^0$	$1.48 \cdot 10^0$
			1000	$0.71 \cdot 10^0$	$2.07 \cdot 10^0$	$2.05 \cdot 10^0$
			10000	$8.04 \cdot 10^0$	$1.98 \cdot 10^0$	$1.99 \cdot 10^0$
	Embedded	EVerRoK-TV	1	$0.40 \cdot 10^{-2}$	$3.45 \cdot 10^0$	$1.50 \cdot 10^0$
			10	$0.54 \cdot 10^{-2}$	$4.91 \cdot 10^0$	$2.75 \cdot 10^0$
			100	$0.14 \cdot 10^{-1}$	$1.01 \cdot 10^1$	$6.21 \cdot 10^0$
			1000	$0.45 \cdot 10^{-1}$	$3.24 \cdot 10^1$	$2.77 \cdot 10^1$
			10000	$0.36 \cdot 10^0$	$4.43 \cdot 10^1$	$4.37 \cdot 10^1$
			100000	$3.07 \cdot 10^0$	-	-
		EVerRoK-Base	1	$0.44 \cdot 10^{-2}$	$3.09 \cdot 10^0$	$0.74 \cdot 10^0$
			10	$0.55 \cdot 10^{-2}$	$4.90 \cdot 10^0$	$0.25 \cdot 10^0$
			100	$0.11 \cdot 10^{-1}$	$1.25 \cdot 10^1$	$1.65 \cdot 10^0$
			1000	$0.16 \cdot 10^{-1}$	$9.29 \cdot 10^1$	$1.13 \cdot 10^1$
			10000	$0.30 \cdot 10^{-1}$	$5.56 \cdot 10^2$	$2.76 \cdot 10^1$
			100000	$0.15 \cdot 10^0$	-	-
	Index	EVerRoK-TV	1	$0.45 \cdot 10^{-2}$	$3.07 \cdot 10^0$	$0.69 \cdot 10^0$
			10	$0.55 \cdot 10^{-2}$	$4.94 \cdot 10^0$	$0.23 \cdot 10^0$
			100	$0.11 \cdot 10^{-1}$	$1.24 \cdot 10^1$	$1.72 \cdot 10^0$
			1000	$0.16 \cdot 10^{-1}$	$9.25 \cdot 10^1$	$1.52 \cdot 10^1$
			10000	$0.29 \cdot 10^{-1}$	$5.60 \cdot 10^2$	$7.48 \cdot 10^1$
			100000	$0.14 \cdot 10^0$	-	-
		RDFTensor	1	$0.13 \cdot 10^0$	$0.11 \cdot 10^0$	$0.12 \cdot 10^{-1}$
			10	$0.22 \cdot 10^0$	$0.12 \cdot 10^0$	$0.20 \cdot 10^{-1}$
			100	$4.95 \cdot 10^0$	$0.28 \cdot 10^{-1}$	$0.20 \cdot 10^{-1}$
		EVerRoK-Base	1	$0.37 \cdot 10^{-1}$	$0.37 \cdot 10^0$	$0.14 \cdot 10^0$
			10	$0.45 \cdot 10^{-1}$	$0.61 \cdot 10^0$	$0.25 \cdot 10^0$
			100	$0.32 \cdot 10^0$	$0.44 \cdot 10^0$	$0.27 \cdot 10^0$
			1000	$1.37 \cdot 10^1$	$0.11 \cdot 10^0$	$0.11 \cdot 10^0$
hard	Embedded	Neo4j	1	$0.48 \cdot 10^{-1}$	$0.60 \cdot 10^0$	$0.42 \cdot 10^{-1}$
			10	$0.76 \cdot 10^0$	$3.47 \cdot 10^0$	$1.81 \cdot 10^0$
		RDFTensor	1	$0.30 \cdot 10^{-1}$	1	$0.39 \cdot 10^{-1}$
			10	$2.61 \cdot 10^0$	1	$0.69 \cdot 10^0$
		EVerRoK-Base	1	$0.25 \cdot 10^{-1}$	$1.02 \cdot 10^0$	$0.89 \cdot 10^0$
			10	$2.87 \cdot 10^0$	$0.91 \cdot 10^0$	$0.92 \cdot 10^0$
		EVerRoK-TV	1	$0.60 \cdot 10^{-2}$	$4.41 \cdot 10^0$	$2.47 \cdot 10^0$
			10	$0.14 \cdot 10^0$	$1.88 \cdot 10^1$	$1.74 \cdot 10^1$
			100	$1.78 \cdot 10^1$	-	-
		EVerRoK-Base	1	$0.70 \cdot 10^{-2}$	$3.85 \cdot 10^0$	$0.23 \cdot 10^0$
			10	$0.17 \cdot 10^{-1}$	$1.56 \cdot 10^2$	$2.90 \cdot 10^1$
	Index	EVerRoK-Base	100	$0.14 \cdot 10^0$	-	-
			1000	$1.82 \cdot 10^0$	-	-
			10000	$8.18 \cdot 10^0$	-	-
		EVerRoK-TV	1	$0.64 \cdot 10^{-2}$	$4.22 \cdot 10^0$	$0.24 \cdot 10^0$
			10	$0.10 \cdot 10^{-1}$	$2.63 \cdot 10^2$	$3.37 \cdot 10^1$
		EVerRoK-TV	100	$0.68 \cdot 10^{-1}$	-	-
			1000	$1.11 \cdot 10^0$	-	-
			10000	$7.22 \cdot 10^0$	-	-

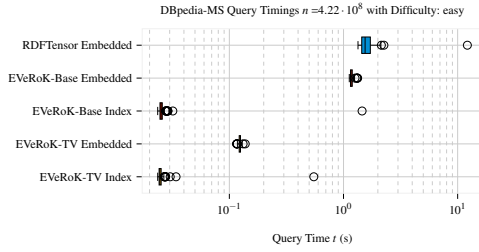


Figure 5: Box plot for the easy query on DBpedia [1]. We compare the baseline (RDFTensor [2]) against our EVerok with and without a vocabulary and with and without indexing. We note that the time t is logarithmic.

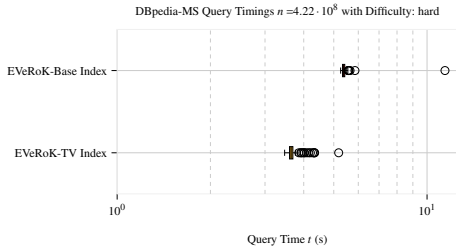


Figure 6: Box plot for the hard query on DBpedia [1]. We show only our EVerok with and without a vocabulary. The other implementation did not complete until a timeout was reached. We note that the time t is logarithmic.

Machine, whereas QLever, our base, uses an efficient C++ implementation. Only the indices for the largest m start to require more memory within our implementation, whereas the other systems, especially Neo4j rises, and RDFTensor stays high even without any load on the large sets. The Fuseki-bases system dips due to timeouts for the last query size. The CPU usage of Figure 4 falls quite flat, since all systems, except the index building, require only one core to operate and cannot take advantage of our full processing power.

DBpedia-Multimodal Search (MS). Moving towards the real-world example within DBpedia reveals similarly tight distributions for both the easy and hard query in Figures 5 and 6. We, again, observe the singular outlier due to the first index build, followed by fast query times for the rest of the queries.

A.2 Resource Usage Acquisitions

The evaluation results also include measurements on the CPU and memory usage of the engines while processing queries. These are acquired every 100 ms during testing and aggregated by taking the median. We use the `psutil`¹ package to acquire the statistics reliable and accurately.

¹<https://pypi.org/project/psutil/>

References

- [1] Jens Lehmann et al. 2015. Dbpedia - a large-scale, multilingual knowledge base extracted from wikipedia. *Semantic Web*, 6, 167–195. <https://api.semanticscholar.org/CorpusID:1181640>.
- [2] 2025. *Representing and querying data tensors in rdf and sparql*. *The Semantic Web: ESWC 2025 Satellite Events*. Springer Nature Switzerland, (Oct. 2025), 62–66. doi:10.1007/978-3-031-99554-5_12.